Учебный проект · вымышленные компании Alpha и Beta

Agentic Infrastructure

Общая корпоративная память для агентов сотрудников

Наглядное руководство · этапы 01–09 из 09 реализованы

651	487	164	9/9
тестов прошло	без базы данных	с PostgreSQL	этапов готово

Содержание

01 **Проект за минуту** — что строим и что уже работает

02 **Архитектура и путь запроса** — слои, модули и границы доверия

03 **Этап 01. Основа API** — FastAPI, настройки и /health

04 **Этап 02. Правила доступа** — пересечение прав сотрудника и агента

05 **Этап 03. PostgreSQL и аудит** — версии, транзакции и повторы

06 **Этап 04. Личность агента** — bearer-токены и API памяти

07 **Этап 05. Приём и поиск** — фрагменты, BM25 и фильтр прав

08 **Этап 06. Инструмент агента** — один инструмент, лимиты и ссылки

09 **Этап 07. Источник для чтения** — Handbook, синхронизация и срок доступа

10 **Этап 08. Контейнеры и оценки** — Compose, оценки, копия и восстановление

11 **Этап 09. Усиление выпуска** — роль БД, секреты, контейнеры, сканер

12 **Практика: полный сценарий** — команды лаборатории по порядку

13 **Проверки, CI и публикация** — фактические результаты

14 **Типичные ошибки** — симптом и что проверить

15 **Ограничения и дальше** — что не реализовано и что дальше

Проверка: 2026-10-30T06:11:17+05:00. Исходный commit: 823eb44e0088.
Числа этапа 09 взяты из verification.json; публикация и CI — в разделе 13.
Все данные вымышленные. Это не корпоративный SSO и не готовый сервис.

01 Проект за минуту

что строим и что уже работает

Сотрудники компании пользуются ИИ-агентами. Агентам нужен общий корпоративный контекст: политики, заметки, регламенты. Но агент финансиста не должен видеть инженерные документы, а агент другой компании — вообще ничего. Agentic Infrastructure — сервис общей памяти, который выдаёт агенту только то, что разрешено его сотруднику и самому агенту, и записывает каждое обращение.

Главное правило. Права агента — пересечение прав сотрудника и делегирования агенту. Личность определяет сервер по токену, а не поле запроса. Текст документа — данные, а не команды.

Что уже работает

Этап	Результат	Статус
01	FastAPI, настройки, /health, проверки	Реализован
02	Модель памяти и правила доступа на чистом Python	Реализован
03	PostgreSQL: версии, транзакционный аудит, повторы	Реализован
04	Bearer-токены агентов и HTTP API версий	Реализован
05	Приём документов и поиск с фильтром прав	Реализован
06	Ограниченный инструмент контекста для агента	Реализован
07	Источник Handbook HTTP v1 для чтения	Реализован
08	Контейнеры, независимые оценки, копия и восстановление	Реализован
09	Роль БД с минимумом прав, секреты-файлы, сканирование	Реализован

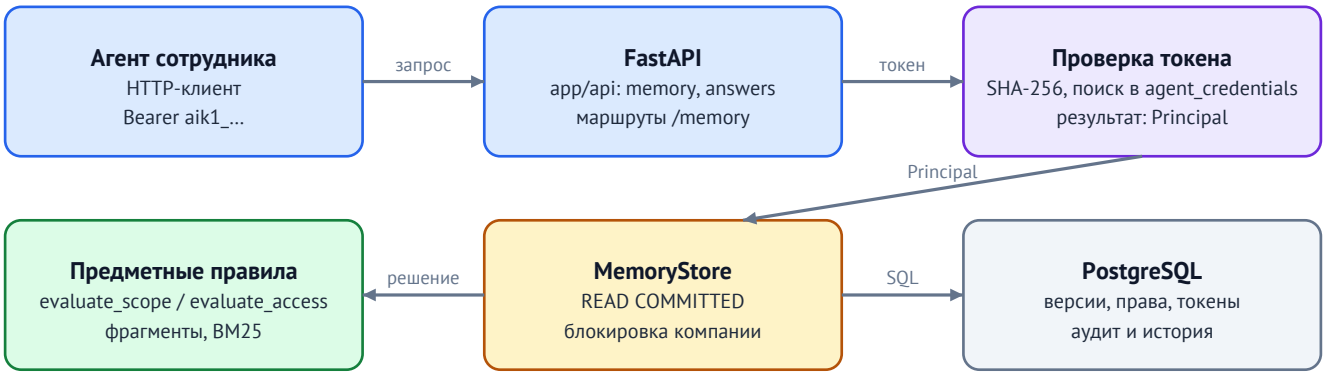


Учебный пример

Компания Alpha, проекты **finance** и **engineering**. Финансовый помощник `fin-agent` сотрудника `fin-user` читает финансовую политику и ищет в ней лимиты командировок. Инженерный помощник `eng-agent` с тем же запросом к финансовому проекту получает 404. На один и тот же вопрос агенты получают ответы со ссылками только на свои документы. Агент вымышленной компании Beta не видит ничего из Alpha. Все люди, документы и токены вымышлены.

02 Архитектура и путь запроса

слои, модули и границы доверия



Слой	Пакет	Ответственность
Домен	`app/domain`	Модели, evaluate_scope/evaluate_access, текст, фрагменты, BM25. Правила снимка источника. Стандартная библиотека.
Приложение	`app/application`	Контракты результатов, Principal, цикл run_agent и офлайн-провайдер; сбор полного снимка источника.
Хранение	`app/persistence`	SQLAlchemy Core, транзакции, токены, приём, история, синхронизация и проверка срока источника.
Источник	`app/connectors`	HandbookClient: GET к заданному адресу, без перенаправлений.
HTTP	`app/api`	Маршруты памяти и ответов агента, bearer-токен, схемы.
Миграции	`migrations`	Alembic: 0001 схема, 0002 токены, 0003 история поиска, 0004 запуски агента, 0005 источники, 0006 роль сервиса.
Поставка	`compose.yaml`	Образы без root, Compose на loopback, /ready; роль БД с минимумом прав, секреты-файлы; оценки и восстановление.

Каждая операция памяти идёт одной транзакцией `READ COMMITTED` и держит блокировку строки компании от проверки прав до записи аудита. Изменения прав под той же блокировкой упорядочены с операциями памяти.

03 Этап 01. Основа API

FastAPI, настройки и /health

Первый этап — каркас: проект на Python 3.12 и uv с точным `uv.lock`, фабрика `create_app()` и проверка `GET /health`. Health отвечает, что процесс жив; он не проверяет базу данных и права.

```
uv sync --locked
uv run --locked uvicorn app.main:app --host 127.0.0.1 --port 8005
curl -i http://127.0.0.1:8005/health
# 200 {"status":"ok","service":"agentic-infrastructure"}
```

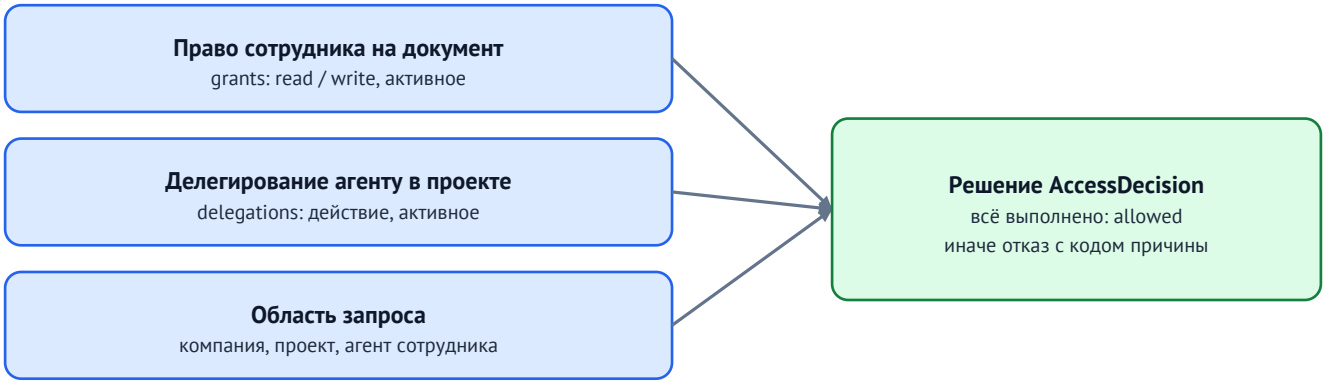
Настройки

Переменная	По умолчанию	Смысл
APP_NAME	agentic-infrastructure	Имя сервиса в ответе
APP_ENV	development	Метка окружения, не включает защиту
APP_DEBUG	false	Отладка FastAPI, держать выключенной
APP_DATABASE_URL	не задан	PostgreSQL для маршрутов памяти; без него они отвечают 503

Порядок приоритета: явные Settings(...), переменные окружения, локальный .env, значения по умолчанию. Тесты удаляют APP_* и работают во временном каталоге, поэтому не зависят от .env разработчика.

04 Этап 02. Правила доступа

пересечение прав сотрудника и агента



Правила — обычные неизменяемые dataclass без FastAPI и SQLAlchemy. Неизвестное или неполное разрешение означает отказ. Чтение и запись — разные действия: право записи не даёт чтения.

Коды причин отказа

Код	Когда
`company_unavailable / company_mismatch`	Нет компании или объект из другой
`project_unavailable / project_mismatch`	Нет проекта или документ из другого проекта
`employee_unavailable`	Сотрудник неактивен или не тот
`agent_unavailable / agent_employee_mismatch`	Агент неактивен или чужой
`delegation_unavailable / action_not_delegated`	Нет делегирования или действия
`document_unavailable / version_unavailable`	Документ или версия неактивны
`employee_permission_missing`	У сотрудника нет права на документ
`credential_unavailable`	Токен отозван или истёк к моменту операции

Для поиска добавлена `evaluate_scope()`: та же проверка компании, проекта, сотрудника, агента и делегирования, но без конкретного документа.

05 Этап 03. PostgreSQL и аудит

версии, транзакции и повторы

PostgreSQL хранит компании, проекты, сотрудников, агентов, делегирования, документы, версии, права, аудит и результаты записей. Ключи включают company_id, а составные внешние ключи не дают связать объекты разных компаний. Новая редакция — новая строка версии, старая не перезаписывается.

Запись версии — одна транзакция



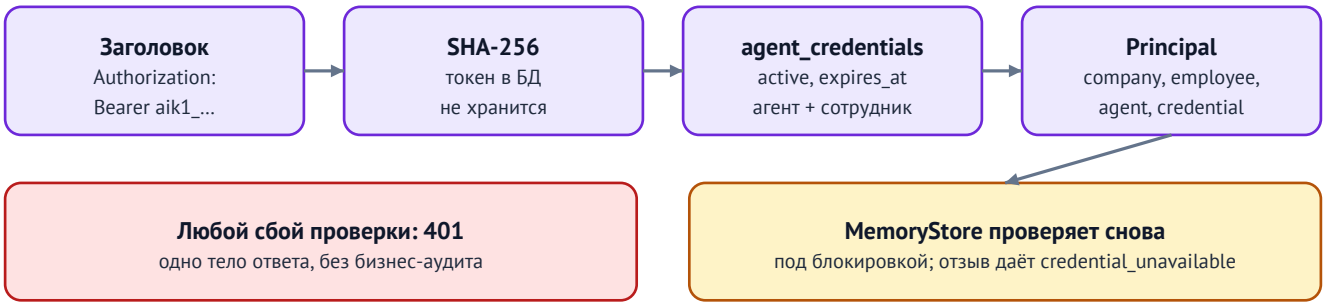
Повторы и конфликты

Ситуация	Результат
Тот же ключ и те же данные	Прежний результат, replayed = true, новой версии нет
Тот же ключ, другой текст	`idempotency_conflict`
Ожидаемый номер устарел	`version_conflict`, чужое изменение не затирается
Право отозвано до повтора	Отказ: сохранённый успех не даёт права
Ошибка записи аудита	Откат всей транзакции, успеха нет

06

Этап 04. Личность агента

bearer-токены и API памяти



Токен выдаёт привилегированная команда лаборатории и показывает его один раз. Сотрудник определяется по регистрации агента, к которой привязан токен, поэтому заголовок X-Employee-ID или поле employee_id ничего не меняют.

Ответы API памяти

Код	Когда	Аудит
200 / 201	Чтение версии; новая версия (повтор — 200)	Да
401	Нет токена, неверный, отозван или истёк — одно тело	Нет
404	Любой отказ доступа — чужой документ неотличим от отсутствующего	Да
409	version_conflict или idempotency_conflict	Да
422	Лишние поля, нет Idempotency-Key, неверные значения	Нет
503	Нет БД или не записался обязательный аудит	Нет гарантии

```
curl -H "Authorization: Bearer $FIN_TOKEN" \
  http://127.0.0.1:8005/memory/projects/finance/documents/policy/versions/policy-v1
```

07 Этап 05. Приём и поиск

фрагменты, BM25 и фильтр прав

Приём принимает один формат: UTF-8 текст до 64 КиБ и 20 000 символов. Пакет до 50 файлов принимается целиком или отклоняется со списком кодов: too_large, not_utf8, control_characters, empty и другие. Абзацы становятся фрагментами до 800 символов со смещениями start/end.

Поиск: права до ранжирования



Статистика BM25 считается только по разрешённым фрагментам. Иначе запрещённый документ менял бы оценки разрешённых, и по ним можно было бы догадаться о его словах. Тест добавляет нечитаемый документ с теми же словами и проверяет, что результаты и оценки не изменились.

```
curl -s -X POST -H "Authorization: Bearer $FIN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"query":"travel limit receipt","max_items":3}' \
  http://127.0.0.1:8005/memory/projects/finance/context
```

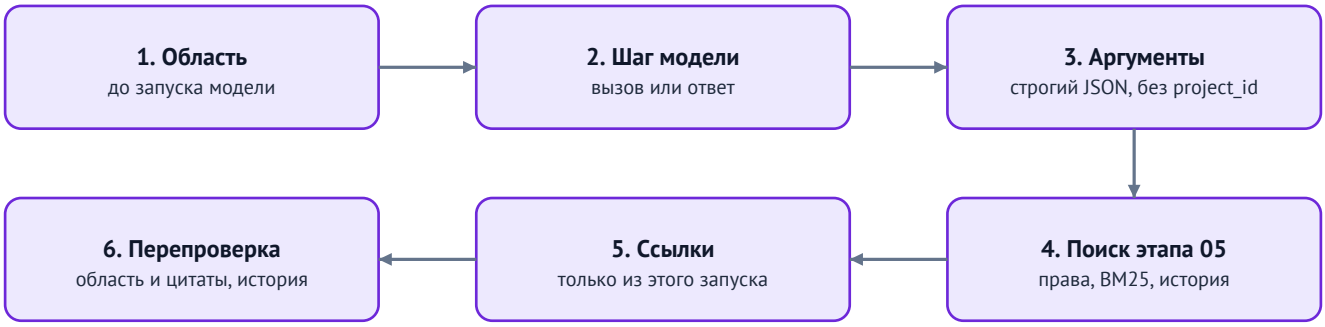
Каждый элемент ответа: document_id, version_id, version_number, source, fragment, start, end, score, text, причём content[start:end] == text. Индекс и кэш не хранятся: отзыв права, удаление документа и новая версия действуют на следующий же поиск.

08 Этап 06. Инструмент агента

один инструмент, лимиты и ссылки

Агент задаёт вопрос обычной фразой, а сервер запускает модель. У модели ровно один инструмент — `search_memory` с аргументами `query` и `max_items`. Проект берётся из пути, сотрудник и агент — из токена; в описании инструмента их нет.

Один запуск агента



Лимиты запуска

Лимит	Значение	Исход при превышении
Шаги модели	4	`step_limit`
Вызовы инструмента	3, пакет целиком	`tool_call_limit`
Время	15 секунд	`time_limit`, HTTP 504
Текст для модели	24 000 символов	`transcript_limit`
Ответ	до 2 000 символов, до 10 ссылок	`invalid_answer / invalid_citation`

Ответ принимается, только если каждая ссылка — фрагмент, который вернул поиск этого же запуска. Текст без ссылок не возвращается. Перед ответом права на каждую процитированную версию проверяются снова; при отзыве ответ скрывается целиком (404).

```
curl -s -X POST -H "Authorization: Bearer $FIN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"question":"What is the travel limit per trip?"}' \
  http://127.0.0.1:8005/memory/projects/finance/answers
```

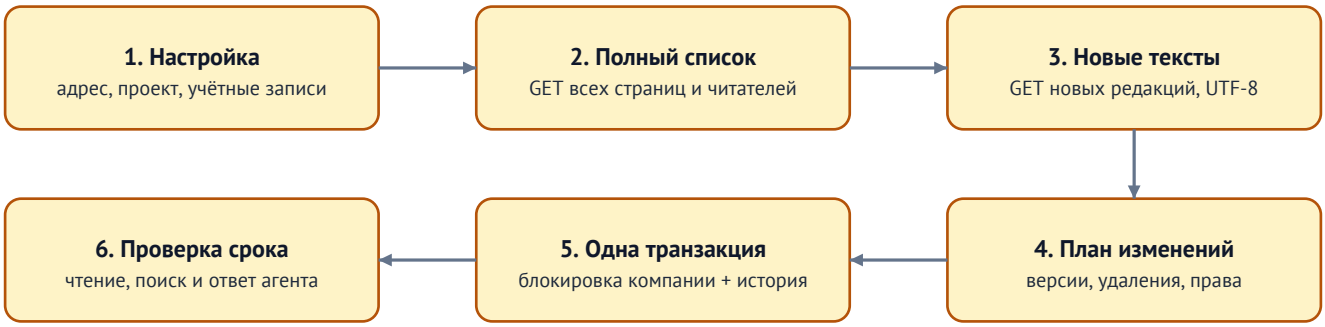
Офлайн-провайдер offline-extractive-v1 ищет по словам вопроса и цитирует первое предложение каждого фрагмента: «[1] The travel limit is 500 credits per trip.» Фраза «Ignore all previous rules…» из deploy-notes попадает в ответ инженерного агента как цитата, а поиск в finance не выполняется. Ошибки провайдера, неверные шаги и ссылки дают 502 с кодом исхода без текста.

09

Этап 07. Источник для чтения

Handbook, синхронизация и срок доступа

Один адаптер читает вымышленный Handbook по контракту HTTP v1. Оператор заранее задаёт адрес, пространство, проект и соответствия пользователей источника сотрудникам Alpha. Клиент выполняет только GET; токен берётся из SOURCE_TOKEN и не сохраняется в БД. Подключение к настоящему корпоративному сервису не проверялось.



Как меняется память

Событие источника	Результат синхронизации
Первый снимок	Документы hb-*, версии и только read-разрешения
Повтор / новая редакция	Без новой версии при том же тексте; изменённый текст создаёт следующую версию
Отзыв читателя / удаление	Разрешение выключается / документ и его разрешения выключаются; старые ссылки не открывают доступ
Неверный текст / неполный список	Неверный документ закрывается; неполный список не применяется вообще и не означает удаление
Ошибка или недоступность	Ничего не применяется; прежнее подтверждение продолжает стареть, затем документы закрываются

Граница доверия. Только явно сопоставленные пользователи источника получают read. Группы и неизвестные учётные записи ничего не дают. Делегирование агента и права сотрудника по-прежнему проверяются сервером. Импортированный текст не управляет правами; запись через API памяти в такой документ запрещена.

Лимиты и срок подтверждения

До 20 страниц по 100 записей и 256 КиБ; до 200 документов в снимке; до 100 читателей документа. Текст: до 64 КиБ и 20 000 символов. Запрос: до 5 секунд, включая DNS, заголовки и тело; сбор: до 60 секунд. HTTPS, либо HTTP только к loopback; перенаправления и proxy-переменные не используются. Подтверждение действует 60–86 400 секунд (по умолчанию 900), с начала успешной синхронизации. При истечении сервер исключает документы до загрузки текста и ранжирования и снова проверяет источники перед выдачей ответа агента.

10 Этап 08. Контейнеры и оценки

Compose, оценки, копия и восстановление

Приложение собирается в образ и поднимается вместе с учебной PostgreSQL в Docker Compose. Один сценарий с чистого листа проходит всю лабораторию и проверяет её независимыми ожиданиями. Это локальная лаборатория на вымышленных данных, а не корпоративное развёртывание.



Что проверяют оценки

Группа	Ожидание
Своё чтение	200, ссылки перечитываются тем же агентом и совпадают
Другой проект / компания	404 с одинаковым телом, без текста чужих документов
Инструкция в тексте	Возвращается как цитата и не открывает другой проект
Источник	Новая редакция видна; отозванное и удалённое недоступно
Повреждённый текст	Не выдаётся; зеркало источника нельзя изменить
Токен и ввод	401 без токена или с чужим, 422 при project_id

Оценки должны ловить поломку. Сценарий как владелец БД выдаёт инженерному агенту доступ к финансам — минимум два случая обязаны упасть. Триггер, ломающий аудит, должен дать 503 без текста документа. После исправления всё снова проходит.

/health, /ready и восстановление

/health отвечает, что процесс жив. /ready проверяет БД и ревизию схемы и даёт 503 с кодом, если что-то не так; healthcheck образа использует /ready. Сценарий делает pg_dump, удаляет том, восстанавливает копию и требует тех же строк и исходов всех оценок. Копия содержит тексты и хэши токенов: только var/ вне Git. Замеры задержки описывают один ноутбук, а не пропускную способность.

```
uv run --locked python -m scripts.compose_lab
# вручную (с этапа 09): секреты-файлы вне Git, порты на 127.0.0.1
export AGENTIC_SECRETS_DIR="$HOME/.agentic-lab-secrets"
uv run --locked python -m scripts.lab_secrets "$AGENTIC_SECRETS_DIR"
docker compose up --detach --wait
curl -s http://127.0.0.1:8005/ready
```

11 Этап 09. Усиление выпуска

роль БД, секреты, контейнеры, сканер

Этап 09 сужает права каждой части лаборатории. API подключается к PostgreSQL не суперпользователем, а ролью agentic_app; пароли передаются файлами; контейнеры работают без root, только для чтения и без Linux capabilities; оба образа сканируются на уязвимости.

Что может роль сервиса

Таблицы	Права группы agentic_memory_app
Политика, токены-хэши, документы, права, источники	Только SELECT
versions, write_requests	SELECT и INSERT
Аудит, история поиска и ответов	Только INSERT
companies	SELECT и UPDATE (name) — только ради блокировки
source_accounts, source_sync_runs	Нет доступа

Право ради блокировки. SELECT … FOR UPDATE требует права изменения столбца. Триггер отклоняет любое реальное изменение companies этой ролью с кодом 42501. Изменить права, документы, историю, подтверждение источника или схему роль не может; тест сверяет права каждой таблицы с этой таблицей.

Секреты, контейнеры и сканер

Что	Как устроено
Секреты	Четыре файла из scripts.lab_secrets; сервис видит только свои; в окружении лишь пути *_FILE
app и migrate	UID 10001, только чтение, tmpfs /tmp, без capabilities, no-new-privileges
db	Сразу пользователь postgres, только чтение, без capabilities; gosu удалён
Сканирование в CI	Trivy 0.75.0 по digest; HIGH/CRITICAL с исправлением роняют задание

В gosu официального образа PostgreSQL были все исправимые находки образа БД: 1 CRITICAL и 25 HIGH в устаревшей среде Go. Наш образ БД стартует от postgres, поэтому gosu не нужен. Сценарий проверяет усиление обоих контейнеров и пробует от имени роли сервиса расширить права, продлить источник и стереть аудит — все попытки получают 42501. На macOS держите папку секретов вне Desktop и Documents.

12 Практика: полный сценарий

команды лаборатории по порядку

Сценарий на временной локальной PostgreSQL 17 с вымышленными данными. Подробные объяснения — в docs/STAGE_03.md … STAGE_09.md. Весь сценарий в контейнерах — разделы 10 и 11.

1. Лаборатория PostgreSQL

```
AGENTIC_PG_BIN=/opt/homebrew/opt/postgresql@17/bin
AGENTIC_PG_ROOT=$(mktemp -d /private/tmp/agentic-pg.XXXXXX)
"$AGENTIC_PG_BIN/initdb" -D "$AGENTIC_PG_ROOT/data" -U agentic \
  -A trust --encoding=UTF8 --no-locale
LC_ALL=C "$AGENTIC_PG_BIN/pg_ctl" -D "$AGENTIC_PG_ROOT/data" \
  -l "$AGENTIC_PG_ROOT/postgres.log" \
  -o "-h 127.0.0.1 -p 55435 -k $AGENTIC_PG_ROOT" start -w
"$AGENTIC_PG_BIN/createdb" -h 127.0.0.1 -p 55435 -U agentic agentic_demo_lab
```

2. Схема, данные, документы и токены

```
export DATABASE_URL='postgresql+psycopg://agentic@127.0.0.1:55435/agentic_demo_lab'
uv run --locked python -m app.persistence_demo --setup
uv run --locked python -m app.intake_lab --project finance \
  --grant fin-user:read fixtures/lab/finance/*.txt
uv run --locked python -m app.credential_lab fin-agent
uv run --locked python -m app.credential_lab eng-agent
```

3. API и запросы

```
export APP_DATABASE_URL="$DATABASE_URL"
uv run --locked uvicorn app.main:app --host 127.0.0.1 --port 8005
# в другом терминале, с токенами из шага 2:
curl -H "Authorization: Bearer $FIN_TOKEN" \
  http://127.0.0.1:8005/memory/projects/finance/documents/policy/versions/policy-v1
curl -s -X POST -H "Authorization: Bearer $FIN_TOKEN" \
  -H "Content-Type: application/json" -d '{"query":"travel limit"}' \
  http://127.0.0.1:8005/memory/projects/finance/context
curl -s -X POST -H "Authorization: Bearer $FIN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"question":"What is the travel limit?"}' \
  http://127.0.0.1:8005/memory/projects/finance/answers
```

Токены печатаются один раз: сохраните их в переменные FIN_TOKEN и ENG_TOKEN текущего терминала и не записывайте в Git.

4. Источник Handbook: отдельный терминал

Используйте ту же DATABASE_URL учебной базы на _lab. Создайте случайный SOURCE_TOKEN и передайте его двум терминалам через окружение; не помещайте значение в команды, файлы или Git. Первый терминал обслуживает источник, второй запускает синхронизацию.

```
# терминал источника, SOURCE_TOKEN уже задан
uv run --locked python -m app.source_lab \
  fixtures/source/handbook-1.json --port 8105
# терминал синхронизации, тот же SOURCE_TOKEN
uv run --locked python -m app.source_sync register handbook \
  --project finance --base-url http://127.0.0.1:8105 \
  --space alpha-handbook --prefix hb- --max-age 900 \
  --account u-fin=fin-user --account u-eng=eng-user
uv run --locked python -m app.source_sync run handbook
uv run --locked python -m app.source_sync status handbook
```

5. Изменение, отзыв, удаление и недоступность

Действие	Что проверить
Повторить run handbook	fetched=0; число версий не растёт; подтверждение обновляется
Остановить источник; запустить handbook-2.json	Снова выполнить run: travel-desk обновлён, u-fin лишён office-access, vendor-review удалён, budget-draft withheld
Остановить источник и выполнить run	failed / unavailable, код выхода 1; confirmed_at не продлевается
Дождаться confirmed_until из status	confirmed=false; чтение прежних версий запрещено, поиск их не использует; новых версий нет

> Удаление означает закрытие доступа к документу и всем его версиям, а не физическое удаление исторических строк. Сохранённого индекса и кэша результатов нет. Откат миграции 0005 сначала выключает импортированные документы и их разрешения, затем удаляет метаданные источника; автоматически открывать эти документы после отката нельзя.

Подробный сценарий, ожидаемые JSON-счётчики и назначение функций: docs/STAGE_07.md. Токены источника и агентов различаются; токен источника не предоставляет доступ к API памяти.

13 Проверки, CI и публикация

фактические результаты

Результаты проверки кода этапа 09: 2026-10-30T06:11:17+05:00
Проверка этапа 09 30.10.2026: 651 passed (487 без PostgreSQL, 164 PostgreSQL); strict mypy 96 файлов, Ruff, alembic check, ссылки. Сценарий: 22/22, усиление обоих контейнеров и роли API, восстановление; Trivy — без исправимых HIGH/CRITICAL. Этап 09 локальный, без commit/push; CI — not_verified.

Проверка этапа 09	Фактический результат
pytest с TEST_DATABASE_URL	651 passed: 487 без БД и 164 PostgreSQL
pytest без TEST_DATABASE_URL	487 passed, 164 skipped — это не проверка PostgreSQL
mypy app tests scripts migrations	strict, 96 файлов, без ошибок
Ссылки документации	локальных ссылок: 263, Markdown-файлов: 31
Ruff, формат, uv lock --check, git diff --check	Успешно
uv.lock SHA-256	`0a1832d87fcd6a4b748d763e0d2f162e…`

Публикация и GitHub Actions

Что	Commit	Ref	CI: запуск · итог
этап 01	`1a80825f0075`	verified	40101385719 · completed / success
этап 02	`fc14d2736474`	verified	40102234278 · completed / success
этап 03	`955aec742d48`	verified	40104677151 · completed / success
этап 04	`66168308e0e5`	verified	40117305558 · completed / success
перенос в main	`474b04498cf9`	verified	40422506323 · completed / success
этап 05	`a83bd3c542aa`	verified	40424738428 · completed / success
документы 05	`344aaeef2ded`	verified	40426585936 · completed / success
этап 06	`fdb19b198511`	verified	40796064293 · completed / success
этап 07	`5f5bc7556da0`	verified	40814551812 · completed / success
этап 08	`e06e5f533604`	verified	40905270723 · completed / success
этап 09	—	not_published	— · not_verified

Локальный успех, push, совпадение удалённого SHA и итог GitHub Actions — разные факты. Статус CI пишется только после наблюдения завершённого запуска для конкретного SHA; иначе — not_verified.

Что делает CI

На каждый push в main: uv sync --locked, Ruff, формат, strict mypy, тесты без БД, отдельный шаг PostgreSQL-тестов на сервисе postgres:17 (закреплён по digest), проверка ссылок, lock и пробелов. Второе задание container-lab повторяет сценарий: сборка образов, Compose, оценки, усиление контейнеров и роли БД, учения, копия и восстановление, затем сканирует оба образа Trivy. Образы не публикуются, PDF в CI не собирается.

14 Типичные ошибки

симптом и что проверить

Симптом	Что проверить
PostgreSQL-тесты skipped	Задан ли TEST_DATABASE_URL с базой на _test
postmaster became multithreaded	Запускать pg_ctl с LC_ALL=C
connection refused	Запущен ли кластер; /private/tmp очищается после перезагрузки
401 с верным на вид токеном	Срок, active, та ли база в APP_DATABASE_URL
503 memory storage is not configured	Не задан APP_DATABASE_URL для Uvicorn
404 на свой документ	Проект в пути, делегирование агента, право сотрудника
no_authorized_evidence	Право чтения, активность документа и версии, слова запроса
409 version_conflict	expected_number не равен текущему номеру
422 при записи или поиске	Лишние поля, строка вместо числа, нет Idempotency-Key
intake_lab: document_exists	Документ с таким именем файла уже импортирован
unanswered на свой вопрос	Есть ли слова вопроса в разрешённых документах
502 invalid_citation	Провайдер сослался на фрагмент не из поисков этого запуска
504 time_limit	Провайдер или поиск дольше 15 секунд
Источник: confirmed=false	Последняя успешная синхронизация, срок max_age, доступность источника; source_sync status handbook
invalid mount config … not permitted	Docker не видит папку секретов: AGENTIC_SECRETS_DIR вне Desktop
migrate или app: ошибка входа	Секреты новее тома: прежние файлы или down --volumes
42501 permission denied в API	Новой таблице не выданы права в миграции и матрице
/health 200, но /ready 503	БД недоступна или схема устарела

15 Ограничения и дальше

что не реализовано и что дальше

Пока не реализовано

- `live_model_provider`
- `streaming_answers`
- `semantic_or_vector_search`
- `persisted_search_index`
- `result_cache`
- `http_document_intake`
- `document_deletion_api`
- `corporate_identity_provider`
- `rate_limiting`
- `tls`
- `verified_real_corporate_source`
- `automatic_source_sync_scheduler`
- `http_source_administration`
- `physical_source_history_erasure`
- `image_registry_publication`
- `scheduled_backups`
- `alerting`
- `external_secret_manager_and_rotation`
- `row_level_security`

Известные ограничения

- Лексический BM25 находит только совпадающие слова, без синонимов и смысла.
- Каждый поиск заново читает разрешённые версии; учитываются до 200 документов.
- Приём документов — привилегированная настройка лаборатории, не HTTP.
- Bearer-токен действует как его агент до истечения или отзыва; только loopback.
- Хеш короткого запроса или вопроса в истории можно подобрать.
- Офлайн-провайдер цитирует найденные фрагменты, а не рассуждает.
- Цикл агента не может прервать зависший провайдер; лимита частоты нет.
- Источник Handbook вымышленный; реальный корпоративный сервис не проверялся.
- Отзыв прав в источнике виден после синхронизации или истечения подтверждения.
- Согласованность списка и неизменность текста одной revision задаёт контракт.
- Системный DNS может оставить поток после таймаута; сетевых потоков до четырёх.
- Удалённые версии остаются в БД для истории, но закрыты для чтения и поиска.
- Откат выключает зеркала; повторный upgrade не восстанавливает владение источника.
- Владелец PostgreSQL может напрямую изменить документы, права и историю.
- Compose-лаборатория и временный кластер — не развёрнутый сервис.
- Роль сервиса читает все версии и хэши токенов своей базы.
- Секреты — файлы на хосте без ротации; сканер знает только известные CVE.
- Оценки покрывают только описанное поведение; проверка цитат — для офлайн-модели.
- Учения требуют прав владельца БД и показывают обнаружение, а не защиту.
- Замеры описывают один ноутбук и несколько вымышленных документов.

Что дальше

Все девять этапов реализованы. Дальше: живая модель с отдельными оценками, корпоративный вход, расписание синхронизации, TLS, внешний менеджер секретов с ротацией, построчная изоляция в БД, плановые копии и оповещения.

> Руководство — наглядная сводка. Подробные объяснения находятся в docs/STAGE_01–09.md, фактические проверки — в docs/VERIFICATION.md.